

Influencer or Observer: Predicting Social Roles

InTweationists Team

Augustin Bouquillard (augustin.bouquillard@polytechnique.edu), Pierre Mazzucotelli (pierre.mazzucotelli@polytechnique.edu), Alexandre Prats (alexandre.prats@polytechnique.edu)

Abstract

This report presents our approach for the "Influencer or Observer" social role classification challenge. Our solution explores models ranging from traditional machine learning techniques to more advanced deep learning models, including transformers. We discuss our models' architectures, hyperparameter tuning strategies, and the performance achieved on the Kaggle test set, measured by the primary metric of accuracy.

1 Machine learning techniques tested

Our initial phase focused on leveraging classical machine learning techniques applied primarily to the metadata features of the dataset. We systematically tested several models, including Logistic Regression, Support Vector Machines (SVM), and various ensemble methods. The ensemble methods explored were Random Forest, Gradient Boosting (GBM), and the more advanced implementations, XGBoost (Chen and Guestrin 2016). This comparative analysis quickly revealed the superior predictive power of the tree-based boosting algorithms, with XGBoost emerging as the most promising standalone model. Building upon this, we attempted to improve performance through model stacking. The rationale was that by combining models with fundamentally different processing approaches (eg. SVM vs. Xgboost), the weaknesses of one model might be compensated by the strengths of another. We experimented with a simple majority voting scheme to aggregate the predictions of the diverse base models. However, this stacked approach ultimately resulted in a degradation of performance, yielding worse results than the best single models (XGBoost) alone. This observation suggests the base models' errors were likely highly correlated or that the simpler stacking mechanism was unable to effectively balance their outputs. At this point, instead of focusing our work on models we decided to focus on the data.

2 XGBoost and feature engineering on the data

The second phase of our methodology shifted the emphasis from model selection to detailed data exploration and feature engineering.

2.1 Extraction and design of non-textual features

To accomplish this, we wrote a function creating new variables across three categories. First, we introduced temporal indicators to describe account maturity and user credibility. These features, which aim to extract more meaningful information than simple date, include metrics like account age, tweet timing or whether the tweet occurred on a weekend.

Second, content measures were used to quantify the composition of each tweet. This involved counting embedded entities such as URLs, hashtags, and user mentions, as well as computing the length of the tweet text and the user's bio. The motivation for these features was the idea that a tweet's composition is dependent on the type of user. Furthermore, these features are relatively simple to compute compared to more advanced techniques like transformer-based embeddings.

Finally, we designed ratios, which are metrics used to uncover relationships within user statistics and serve as proxies for user influence and engagement. Key examples include the followers-to-friends ratio and the tweets per day count.

The engineering of new, non-textual features significantly boosted the model's performance, validating our hypothesis that these metadata signals are crucial for distinguishing between Influencers and Observers. So we tried to add others features, as discussed below.

2.2 Adding a semi-supervised feature

By applying unsupervised methods (k-means in our case) on the existing numerical features we made the assigned cluster into a new feature in the set. The idea behind these features was that it would be possible to apply our routines to all the available data (train set + Kaggle test data) before the training of our models and as input during the inference stage. This allows us to catch some richer information from the Kaggle test.

Although the addition of the k-means cluster feature did not lead to a measurable improvement in final accuracy, this approach was included in the report to illustrate the exploration of a semi-supervised technique, which could be valuable for its potential to leverage latent structural information within the data, complementing supervised approaches well.

2.3 Meaningful text representation

Beyond metadata, the treatment of the textual component of tweets was also important to extract lexical, semantic, and contextual information that captures how users communicate. To this end, we explore a spectrum of text representations, from lightweight lexical features to more advanced transformer-based embeddings.

2.3.1 Lexical and semantic representations

The treatment of text began with the extraction of lexical and semantic representations. This phase included utilizing TF-IDF (Sparck Jones 1972), a representation that highlights discriminative words based on their frequency patterns. Furthermore, sentiment analysis was applied to extract polarity and subjectivity scores, characterizing each tweet's emotional tone. Finally, embedding models, specifically Word2Vec (Mikolov et al. 2013) and FastText, were used to generate semantic representations for each tweet.

2.3.2 Contextual embeddings and transformer-based features

Beyond basic lexical representations, we explored modern techniques utilizing contextual embeddings. We initially encoded tweets using pretrained transformer models, such as BERT and RoBERTa (we even tried CamenBERT which is better in french, the language of the tweets). These embeddings are valuable because they capture nuanced linguistic patterns, word disambiguation, and long-range dependencies that traditional methods like TF-IDF often miss. For improved task alignment and specialization in identifying user intent, we also explored fine-tuning transformer models. The strategic decision to use LoRA (Low-Rank Adaptation from Hu et al. 2022) for this purpose was primarily driven by the need for computational efficiency, allowing us to accelerate training while minimizing the memory footprint and time required for experimentation.

2.4 Results

By doing all of this feature engineering and then optimizing the hyperparameters and the regularization of our XGBoost through Gridsearch/RandomSearch, we get 0.84 of accuracy. This pipeline is illustrated Figure 1a.

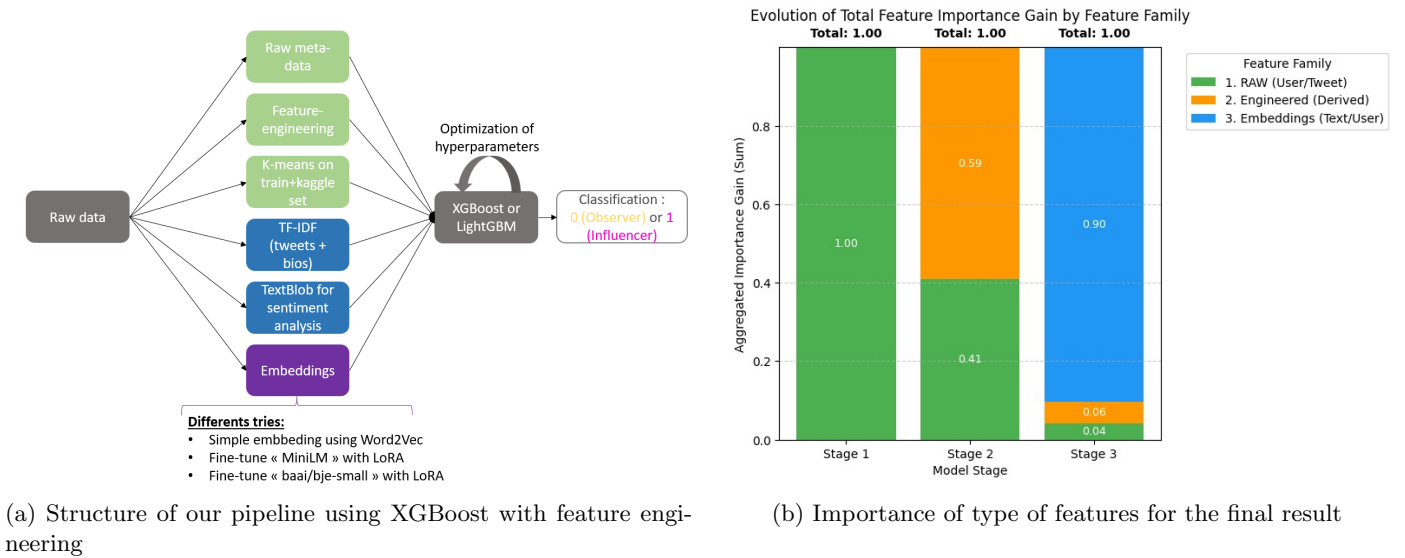


Figure 1: Xgboost approach. (a) Pipeline's structure. (b) Interpretability analysis.

Finally, by adding a majority vote on all tweets from each user after all the previously presented steps, we get an accuracy of 0.843.

2.5 Interpretability: feature importance analysis

To understand which factors the XGBoost model deemed most critical for classifying users as 'Influencer' or 'Observer,' we computed the feature importance using the gain metric. This approach allows us to interpret the model's decisions and validate our feature engineering strategy across three distinct stages, the importance of which is visualized in Figure 1b. Stage 1 (baseline) involved training on only the initial numerical features. Stage 2 (engineered features) added custom-engineered features (described in Sections 2.1 and 2.3.1), and Stage 3 (stage 2 + contextual embeddings described in 2.3.2) included text embedding features. The engineered features added in Stage 2 demonstrated an importance quite similar to that of the initial raw data, and critically, Stage 2 yielded the best classification accuracy. Conversely, when the text embedding features were included in Stage 3, they became predominant in terms of gain importance, yet this resulted in a slightly worse accuracy. One hypothesis to explain this degradation is that the high importance of the embedding features may have diluted the influence of the more relevant, discriminating features present in Stage 2.

3 Dual-path deep learning

Seeing that we were plateauing on the leaderboard with our XGBoost method, we tried a more traditional approach using a deep learning model that processes both metadata and text simultaneously through a dual-path design before

fusing the learned representations for final classification.

3.1 Dual-path feature encoding : a deep learning approach

The model begins by processing the distinct feature types through specialized encoders:

- **Text encoding:** The representations of the tweet text were processed by a sequence-processing encoder adapted from the transformer architecture. Indeed, the transformer (Vaswani et al. 2017), leveraging its self-attention mechanism, can capture complex linguistic relationships.
- **Metadata encoding:** The augmented (as viewed previously) numerical metadata features (e.g., account age, follower ratios, status counts) were passed into a standard Multi-Layer Perceptron (MLP).

3.2 Fusion classifier (MLP)

Finally, the output vectors generated by the text encoder and the metadata encoder were concatenated into a single vector. This combined vector was then passed into a final classification layer for prediction, illustrated in Figure 2. As seen in part 2, we also add a majority vote on each user on top of the whole classifier to get a little extra boost in terms of accuracy to finally get an accuracy of 0.837 on the kaggle data set. The slightly worse accuracy of the Dual-Path Deep Learning model (83.7%) compared to the optimized XGBoost (84.3%) may be attributed to the simplicity of the Fusion Classifier (MLP), which may have struggled to optimally integrate the diverse representations, or a lack of sufficient hyperparameter tuning due to computational constraints.

We prevented overfitting using two main methods: early stopping, which halted training when validation performance ceased improving, and dropout, which randomly deactivated neurons during training to enforce the learning of robust features and improve generalization.

3.3 Combining dual-path and XGBoost approaches

We attempted to combine the Dual-Path Deep Learning and XGBoost methods, as illustrated in Figure 3. The rationale was that the distinct nature of the two models could allow them to mutually compensate for individual weaknesses and improve overall predictive power. Unfortunately, we get an accuracy of only 0.836.

4 Conclusion and perspectives

This project underscored the critical importance of effective data analysis and feature engineering, since our models' performances were inherently tied to extracting relevant signals from both textual and behavioral metadata. As this was a first exposure to a "real-world" use case, we faced significant computational time constraints during model optimization, necessitating practical compromises. We also learned that an incremental working strategy, focused on understanding the impact of each modification, was more conducive to robust model building than blind or too broad experimentation. Finally, we were surprised by the highly competitive efficiency of tree-based boosting algorithms, such as XGBoost, relative to wider deep learning architectures, as exposed in the following table:

Looking ahead, it would be interesting to explore several avenues to push the model's performance and efficiency. First, we need to significantly improve our computational efficiency to reduce training time. Doing so will allow us to test far more combinations of hyperparameters, which could unlock a more optimal configuration for our current architecture.

Furthermore, it would be highly pertinent to explore more sophisticated deep learning architectures such as Graph Neural Network (GNN), if we had more info about user relationships : social media are modeled well by graph structures, where users are nodes and their interactions form the edges. Our current hybrid model treats each user in relative isolation; a GNN could explicitly model complex propagations and relational patterns within the network.

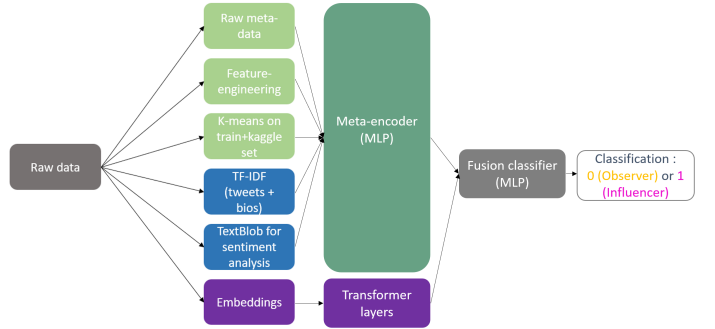


Figure 2: Dual-path deep learning approach with a final fusion classification

Classification model	Accuracy
Dummy model (most frequent)	53.4%
Naive LogisticRegression	62.5%
Best XGBoost	84.0%
Best XGBoost with majority vote	84.2%
FFNN only	83.1%
Best dual-path deep learning	83.7%
Combining dual-path and xgboost	83.6%

Table 1: Comparison of the accuracy performance of different models on the Kaggle test set.

5 Annexes

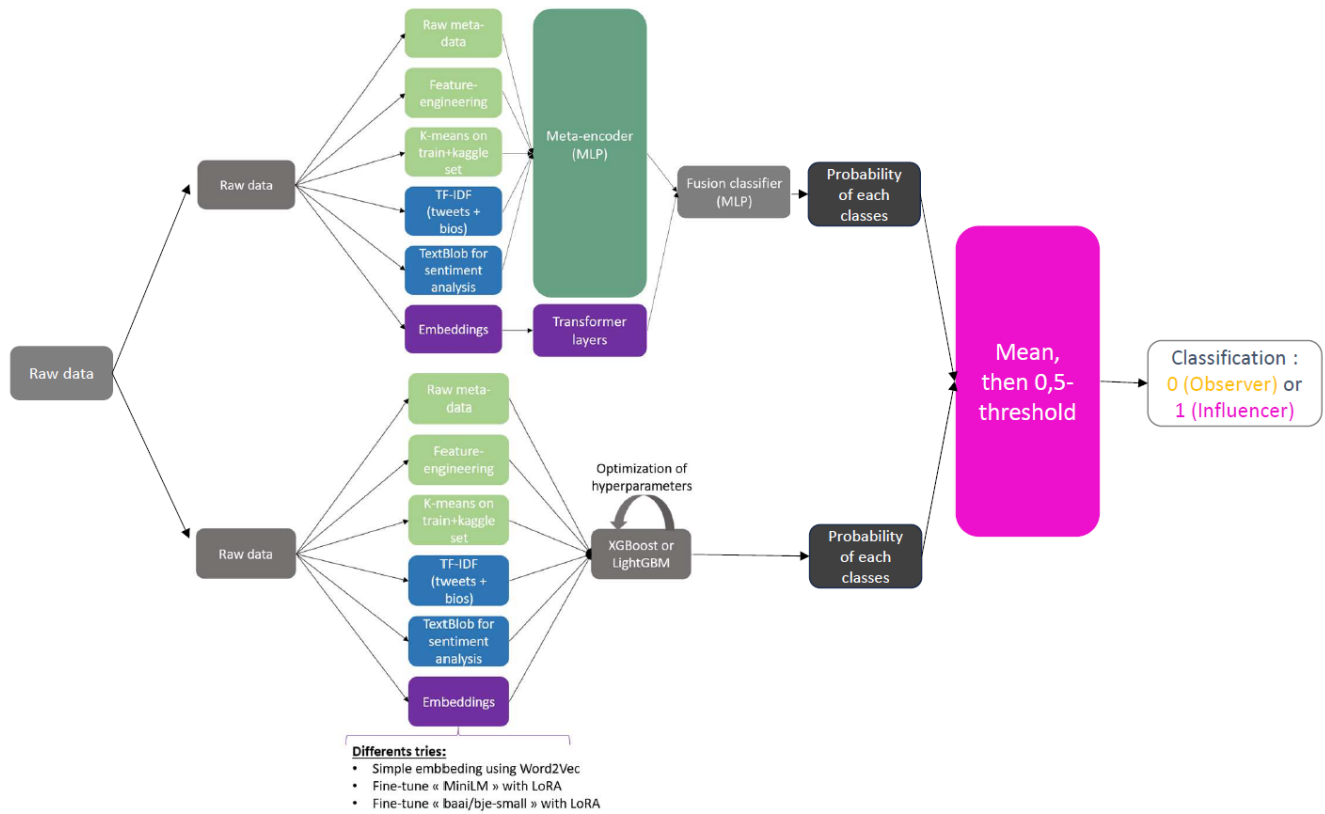


Figure 3: Combination of the Dual-path deep learning approach with the XGBoost one

References

- [1] Tianqi Chen and Carlos Guestrin. “XGBoost: A scalable tree boosting system”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM. 2016, pp. 785–794.
- [2] Edward J. Hu et al. “LoRA: Low-rank adaptation of large language models”. In: *International Conference on Learning Representations*. 2022. DOI: 10.48550/arXiv.2106.09685.
- [3] Tomáš Mikolov et al. “Efficient estimation of word representations in vector space”. In: *arXiv:1301.3781* (2013).
- [4] Karen Sparck Jones. “A statistical interpretation of term specificity and its application in retrieval”. In: *Journal of Documentation* 28.1 (1972), pp. 11–21. DOI: 10.1108/eb026526.
- [5] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems* 30. 2017, pp. 5998–6008. DOI: 10.48550/arXiv.1706.03762.